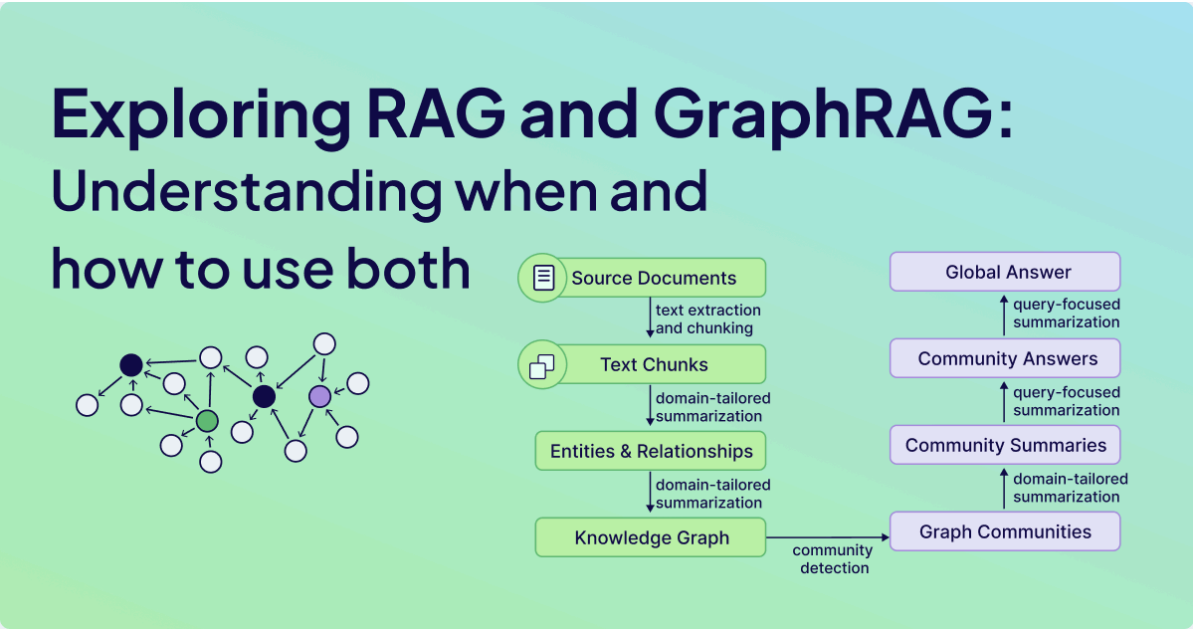


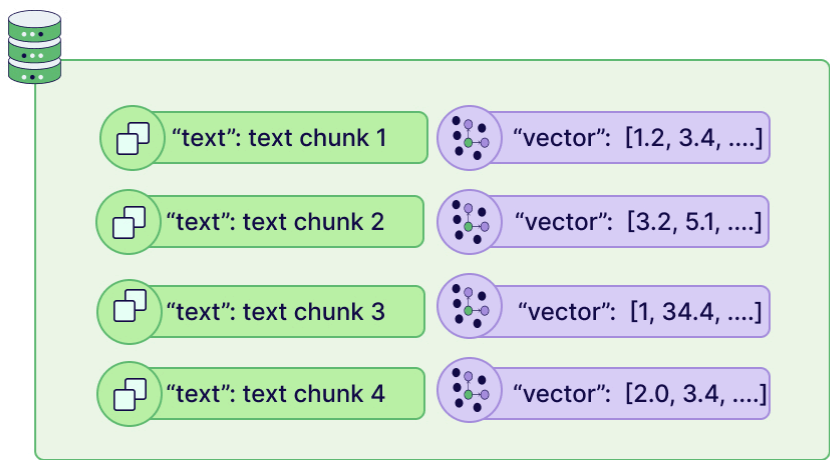
GraphRAG 全流程解析与实战



一、前言：为什么我们需要 GraphRAG?

你有没有遇到过这种情况：问大模型一个很专业的问题，它回答得头头是道，但其实全是错的，也就是一本正经地胡说八道，即大模型的幻觉现象。出现这种现象的原因是大模型的知识是静态的，它依赖于训练数据，而无法实时地获取训练之外的数据。

为了解决幻觉问题，RAG 诞生了，它的思路简单来说就是先检索再生成，即先从知识库中检索出相关内容，再把这些结果和问题一起交给大模型生成答案。



而 Naive RAG 的缺陷也很明显，其只能处理“一步到位”的问题，无法处理**多跳推理**或**证据链完整性（实体关联）**的问题。这里举个简单的例子：

知识库内容：

文档 1：小明的爸爸是老王。

文档 2：老王的女儿是小红。

文档 3：小红是北京大学的学生。

文档 4：.....

问题：

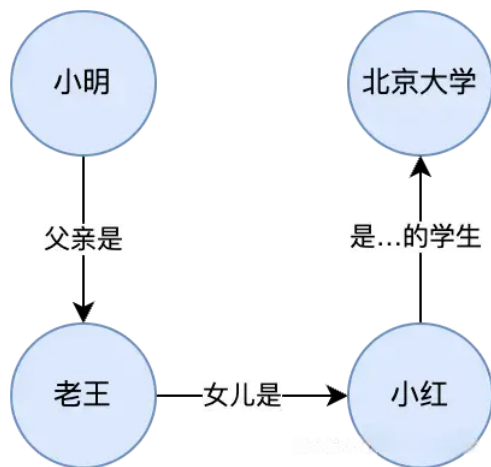
小明的爸爸的女儿在哪里上学？

这个问题实际需要 3 步推理：

1. 小明的爸爸是老王（从文档 1 获得）
2. 老王的女儿是小红（从文档 2 获得）
3. 小红在北京大学上学（从文档 3 获得）

对于 Naive RAG，由于存在相似度阈值，一次检索时一般很难把三条文档全部都找到，所以大模型无法把这些信息都串起来，最终依旧会胡说八道。为了解决这一问题，GraphRAG 诞生了，它将知识图谱引入 RAG 流程，把知识表示成“节点 + 边”的形式，把不同文档中的实体及其关系连接起来。在查询时，不仅会查询出相似度最高的文档，还会基于知识图谱查询出与其关联的文档，因此大模型就能够拥有更加完整的上下文来回答问题。

以上面的例子作为示例，在 GraphRAG 中会存在如下的知识图谱：



二、知识图谱（KG）核心概念

知识图谱（Knowledge Graph）是人工智能和数据科学领域中非常关键的基础设施。简而言之，它是一种用图结构（Graph）来描述客观世界中概念、实体及其关系的技术。

知识图谱 = 用“图”的方式，把现实世界的对象（实体）及其关系（语义关系/属性）结构化表示出来，并支持查询、推理、融合与持续演化的知识库。

2.1 基本介绍

2.1.1 定义

知识图谱本质上是一种**语义网络（Semantic Network）**。它将现实世界中的事物（实体）作为节点，将事物之间的联系（关系）作为边，构成一张巨大的网。

- **口号：**“Things, not strings” (是事物，而不是字符串)。这是Google在2012年提出知识图谱时的核心理念，旨在让机器像人类一样理解“词语背后的含义”。

在知识图谱里，信息通常以“**节点-边**”的方式组织：

- **节点 (Node)**：代表一个实体/概念（人、公司、药品、订单、地点、论文、设备.....）
- **边 (Edge)**：代表实体之间的关系（任职于、位于、生产、依赖、相似、同义、属于.....）
- 节点/边还可以带**属性 (Properties)**：比如姓名、日期、金额、置信度、来源等（看你采用的图模型）

这种组织方式的优势在于：当你关心的是“**连接**”与“**路径**”（例如**多跳推理**、**关联分析**、**血缘追踪**、**关系网络**）时，**图模型比表模型更自然**。

2.1.2 核心价值

- **结构化**：将非结构化的文本（如新闻、网页）转化为计算机可处理的结构化数据。
- **关联性**：打破数据孤岛，建立数据之间的连接。
- **可解释性**：相比于深度学习的“黑盒”，知识图谱的推理过程是基于逻辑的，具有很强的可解释性。

2.2 核心组件与数据模型

知识图谱的数据模型主要由三元组 (Triples) 构成，这是最小的知识单元。

2.2.1 三元组 (Triple)

由 (头实体, 关系, 尾实体) 或 (实体, 属性, 属性值) 组成。

- **例子**：（李白，朝代，唐朝）
 - **实体 (Entity/Node)**：图中的节点。代表具体的事物，如“李白”、“唐朝”、“苹果公司”。
 - **关系 (Relation/Edge)**：连接节点的边。代表实体间的互动，如“出生于”、“位于”、“属于”。
 - **属性 (Attribute)**：实体的特征。如李白的“出生年份：701年”。
- **主谓宾**：谓语所代表的关系在主语与宾语之间成立
 - 自然语言 -> 三元组
 - 小明的职业是物理学家
 - 小明，职业，物理学家

2.2.2 本体 (Ontology) / 模式层 (Schema)

如果说实体是数据，那么本体就是**数据的“模具”或“规则”**。它定义了知识图谱中包含哪些类 (Class) 以及类之间的层级关系。

- **类 (Class)**：如“人物”、“城市”、“书籍”。
- **层级**：“作家”是“人物”的子类。
- **作用**：本体层约束了数据的逻辑，例如规定“出生地”关系的尾实体必须是“地点”类，而不能是“时间”。

2.3 技术架构与构建流程

构建一个知识图谱通常需要经历以下步骤：

2.3.1 数据获取

数据源通常分为三类：

- **结构化数据**：数据库、CSV文件（最容易处理）。
- **半结构化数据**：百科Infobox、网页表格。
- **非结构化数据**：纯文本、新闻报道、语音（处理难度最大，需结合NLP技术）。

2.3.2 知识抽取 - 核心难点

从非结构化文本中提取三元组的过程。

- **实体识别 (NER)**：识别出文本中的“马云”、“阿里巴巴”。
- **关系抽取 (RE)**：识别出“马云”和“阿里巴巴”之间是“创始人”关系。
- **属性抽取**：提取“马云”的“年龄”、“籍贯”。

2.3.3 知识融合

解决多源异构数据冲突的问题。

- **实体对齐 (Entity Alignment)**：确认“C. Ronaldo”、“C罗”和“克里斯蒂亚诺·罗纳尔多”是指同一个人。
- **指代消解**：解决文本中“他”、“那个公司”具体指代谁的问题。

2.3.4 知识存储

- **RDF存储**：适合学术界，强调语义标准。
- **图数据库 (Graph Database)**：适合工业界，查询效率高。代表产品：Neo4j, JanusGraph, Nebula Graph。

2.3.5 知识推理

基于已有的知识推导出新的知识。

- 例子：已知 A是B的父亲， B是C的父亲 -> 推理出 A是C的爷爷。

2.4 应用场景

知识图谱在工业界的应用非常广泛，主要解决“连接”和“认知”的问题。

2.4.1 智能搜索

- **场景**：在Google或百度搜索“刘德华的妻子”，直接展示答案卡片，而不是仅仅给出包含这些关键词的网页链接。
- **原理**：搜索引擎理解了实体关系，直接在图谱中检索（刘德华，妻子，？）的结果。

2.4.2. 推荐系统

- **场景：** 抖音、淘宝的“猜你喜欢”。
- **优势：** 提供**可解释的推荐**。
 - **传统推荐：** “买过A的人也买了B”。
 - **图谱推荐：** “推荐给你电影B，因为你喜欢电影A，而A和B的导演是同一个人，且题材都是科幻。”

2.4.3. 金融风控

- **场景：** 反欺诈、反洗钱。
- **原理：** 利用图算法（如连通分量、PageRank）发现复杂的关联网络。例如，多个看似无关的公司共用同一个电话号码、同一个担保人，在图谱中会形成明显的**环状结构**或**聚集**，从而识别出欺诈团伙。

2.4.4. 智能问答与客服

- **场景：** Siri, 小爱同学, 银行自动客服。
- **原理：** 将用户的自然语言问题转化为图查询语言（KBQA），精准返回事实性答案。

2.4.5. 医疗健康

- **场景：** 辅助诊疗、药物研发。
- **原理：** 构建 **（疾病，症状，药物）** 图谱。当医生输入一组症状时，系统通过图谱推理出可能的疾病，并推荐相关文献中验证过的药物，发现药物之间的潜在副作用（药物相互作用）。

2.4.6. 供应链管理

- **场景：** 风险传导分析。
- **原理：** 构建产品组件图谱。当某个细小零件的供应商（如某地的芯片厂）发生火灾，系统能顺着图谱路径，瞬间计算出哪些下游整车厂会受到影响，从而提前预警。

2.5 总结

知识图谱是让机器从“感知”（看图、听音）进化到“认知”（理解、思考）的关键技术，是通往强人工智能（AGI）的必经之路。

三、GraphRAG 流程解析

3.1 GraphRAG 介绍

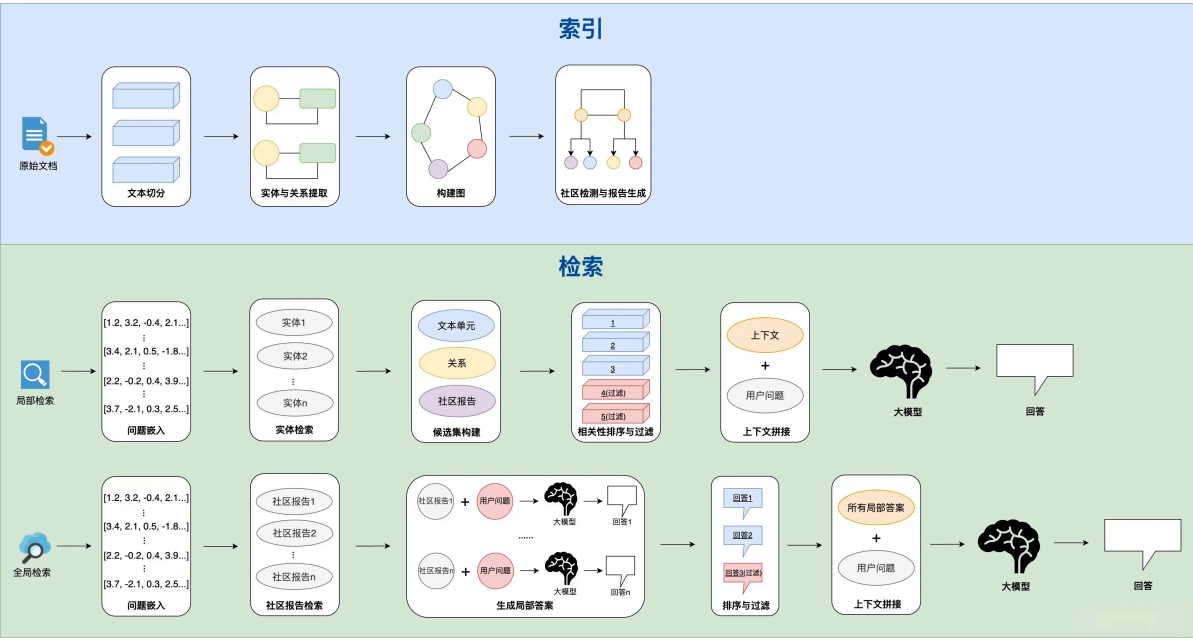
GraphRAG 是一种结构化、分层的检索增强生成 (RAG) 方法，与使用纯文本片段的朴素语义搜索方法形成对比。GraphRAG 过程包括从原始文本中提取知识图谱，构建社区层次结构，为这些社区生成摘要，然后在执行基于 RAG 的任务时利用这些结构。

微软文档：[欢迎 - GraphRAG 文档](#)

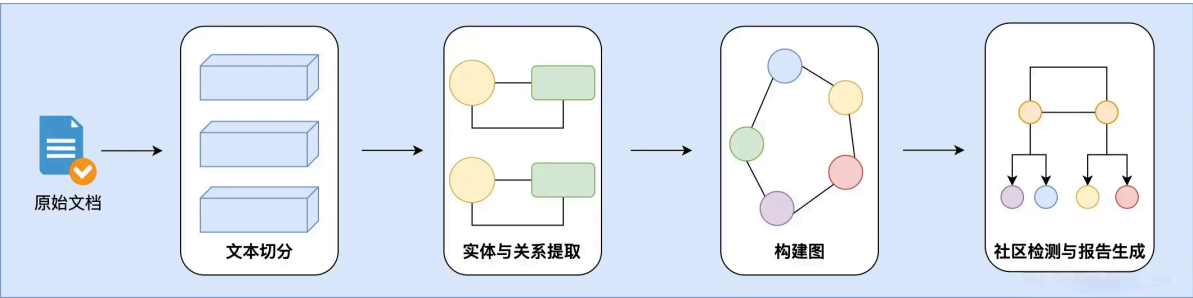
Github：[microsoft/graphrag: A modular graph-based Retrieval-Augmented Generation \(RAG\) system](#)

与传统 RAG 类似，GraphRAG 的流程总体上也可以分为索引和查询，但二者在实现细节上存在很大的区别。

GraphRAG 的详细流程如下图所示：



3.2 索引



为了更清晰地叙述每个索引流程，这里使用一个简单的示例来贯穿所有的流程来讲解，示例如下：

在2025年春季科技大会上，知名人工智能研究员小明与清北京大学计算机学院合作，发布了一款名为“知澜”的多模态大模型。该模型在自然语言处理、图像识别以及语音交互领域均表现出色，能够实现文本、图像和语音信息的无缝理解与生成，支持跨模态知识融合与智能推理。

3.2.1 文本切分

文本切分指的是将原始文本拆分成更小的、可分析的文本单元（TextUnits），文档切分的质量将直接决定了后续检索的准确性和大模型回答的效果，且能够避免超过大模型上下文窗口大小的问题，这一步与传统 RAG 是类似的。

常见的切分策略如下：

名字	描述	优点	缺点
固定大小拆分	指定字数或 token 数来切分	实现简单、速度块	可能会割裂语句，打断语义完整性

名字	描述	优点	缺点
结构拆分	基于文档格式（如 Html、Markdown）拆分，本质上是借助这些文档特有的格式来拆分语句	可以保留原始文档的结构逻辑，语义完整度高	依赖于结构清晰、规范的文档，对于无结构的语句（如纯文本）无法使用
语义拆分	根据语义边界（如段落、句子、主题变化等）拆分，可以采用 NLP 方法，如如分句、主题检测或 Embedding 聚类	最符合人类理解，可保留语义一致性	实现复杂，需要依赖 NLP 模型、Embedding 计算或聚类 等，且计算开销大，效率低
递归拆分	先按大分隔符（如段落）拆分，再按句子等进行拆分，直到拆分得到的块满足长度限制	在语义完整性和长度控制两个度量之间保持平衡	设计较复杂，需要合适的层级和递归停止条件

以固定 40 token 大小切分策略演示，上面的示例可以切分成如下文本单元表：

id	human_readable_id	text	n_tokens	document_id
s8df60f2f66c3	1	在2025年春季科技大会上，知名人工智能研究员小明与清北大学计算机学院合	40	[6db8fe3d86d]
a2b7b0e354fd	2	清北大学计算机学院合作，发布了一款名为“知澜”的多模态大模型。该模型在自然语言处理	40	[6db8fe3d86d]
256254a9806f	3	。该模型在自然语言处理、图像识别以及语音交互领域均表现出色，能够实现文	40	[6db8fe3d86d]
a723cbe34299	4	现出色，能够实现文本、图像和语音信息的无缝理解与生成，支持跨模态知识融合	40	[6db8fe3d86d]
21232021dc6c	5	模态知识融合与智能推理。	17	[6db8fe3d86d]

- id：全局唯一标识符。
- human_readable_id：便于人类查看的序号，相比 id 会更加直观。

- text: 文本单元对应的原始文本。仔细看的话会发现相邻文本单元中出现了重叠的情况，原因是 GraphRAG 采用了重叠窗口切分策略，即在相邻文本单元之间保留一部分重叠内容，以避免语义被硬性截断、保证实体与关系的完整抽取，并在后续检索时提升召回率和准确性。
- n_tokens: 文本单元的 token 总数。
- document_ids: 表示文本单元来自于哪些文档列表。

3.2.2 实体与关系提取

实体:
文本中可以作为“独立对象”存在的东西。它往往是名词或名词短语，比如人物（马斯克）、组织（微软）、地点（北京）、事件（世界杯）、甚至抽象概念（人工智能）。

关系:
不同实体之间的联系。它通常表现为动词、介词或逻辑关系，比如“马斯克「创立」特斯拉”，“微软总部「位于」西雅图”。

文本切分后，GraphRAG 会使用大模型提取出每个文本单元中的实体与关系，并建立实体表和关系表，这样可以让信息的逻辑关系更清晰，方便后续检索、推理和知识图谱构建。

上面的示例的实体如下所示：

id	human_readable_id	title	type	description	text_unit_ids
cf196a73	0	2025 年春季科技大会	事件	2025 年春季人工智能研究员小明与清北京大学计算机学院合作举办的科技大会	[8df60f2f66c3]
e9933f7b	1	小明	人物	一位在 2025 年春季科技大会上发表演讲或参与合作的知名人工智能研究员	[8df60f2f66c3]
d0b0b927	2	清北京大学计算机学院	组织	与人工智能研究员小明合作参加 2025 年春季科技大会的清北京大学计算机学院	[8df60f2f66c3]
50d0eff5	3	知澜	项目	是一款多模态大模型，由清北京大学计算机学院合作发布	[a2b7b0e354fd]

- id: 全局唯一标识符。
- human_readable_id: 便于人类查看的序号，相比 id 会更加直观。
- title: 实体名。

- type：实体类别。
- description：实体描述。
- text_unit_ids：表示实体来自于哪些文本单元列表。

关系如下所示：

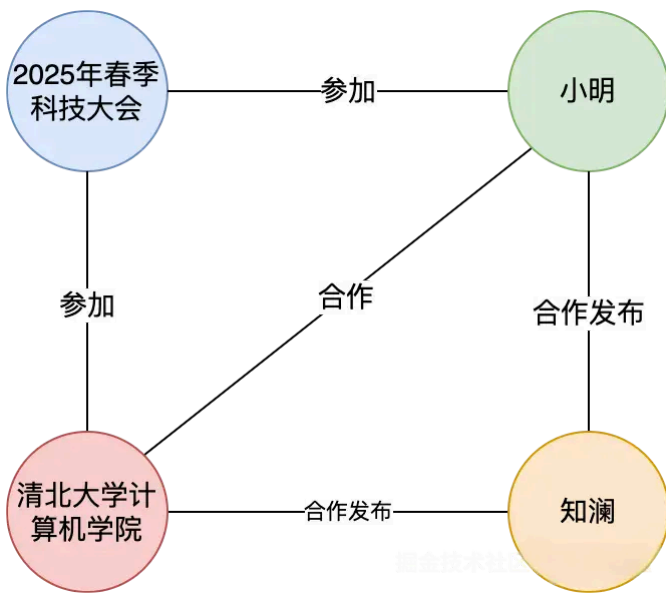
id	human_readable_id	source	target	description	text_unit_ids
c7c70281	0	2025年春季科技大会	小明	小明参加了2025年春季科技大会，是该大会的参与者，并可能担任演讲者	[8df60f2f66c3]
ad37582e	1	2025年春季科技大会	清北京大学计算机学院	清北京大学计算机学院参加2025年春季科技大会	[8df60f2f66c3]
ea303919	2	小明	清北京大学计算机学院	小明在活动现场与清北京大学计算机学院合作	[8df60f2f66c3]
688cada1	3	清北京大学计算机学院	知澜	清北京大学计算机学院合作发布了“知澜”多模态大模型	[a2b7b0e354fd]
4d4ef204	4	小明	知澜	小明合作发布了“知澜”多模态大模型	[8df60f2f66c3,a2b7b0e354fd]

- id：全局唯一标识符。
- human_readable_id：便于人类查看的序号，相比 id 会更加直观。
- source：关系的起点实体。
- target：关系的终点实体。
- description：关系描述。
- text_unit_ids：表示关系来自于哪些文本单元列表。

3.2.3 构建图

构建图指的是将前一步抽取出的实体和关系组织成知识图谱的过程，每个实体作为图中的节点，实体间通过关系形成边，从而把文本信息结构化为图形表示，这样不仅保留了原始文本的语义，还便于后续进行图搜索和关系推理。

上面的示例的图结构如下所示：



3.2.4 社区检测与报告生成

构建完图后，GraphRAG 会对其构建的图进行结构化分析与信息整理。

首先会进行社区检测，即通过分析图中节点（实体）与边（关系）的连接密度，将图划分为若干社区（社区指的是图结构中紧密关联的节点集合），每个社区内部的节点连接紧密、相关性高，而不同社区之间的连接相对稀疏。具体来说，微软 GraphRAG 使用 **Leidan 聚类算法**来完成社区检测的动作。社区检测的核心价值在于降低图查询的复杂度，并揭示知识图中的热点区域和关键关系，为后续检索和知识分析提供基础。

在完成社区划分后，会生成社区报告，将社区检测结果转化为可读的分析报告。报告通常包含社区概览、核心节点与关系统计、社区间关系分布、可视化图表以及文本摘要等内容。通过统计每个社区的节点数量、关系密度及主要实体，并结合可视化工具展示社区结构图或热点节点图，用户可以直观理解知识图的结构和信息分布，进而辅助知识发现和决策分析。

上面的示例检测出的社区如下所示，由于示例只有一段文本，因此最终只存在一个社区。

id	human_readable_id	community	level	title	entity_ids	relationship_ids	text_unit_ids
5ebd0d55	0	0	0	社区 0	[cf196a73,e9933f7b,d0b0b927,4d4ef204]	[c7c70281,ad37582e,ea303919,688cada1,4d4ef204]	[8df60f2f66c3,a2b7b0e354fd]

- id：全局唯一标识符。
- human_readable_id：便于人类查看的序号，相比 id 会更加直观。
- community：社区编号。
- level：层级，指的是社区在知识图谱中的深度，值越大表示在划分层次中处于更细粒度的层级，如 level = 1 是 level = 0 的子社区。
- title：社区的简短标题。
- entity_ids：属于社区的实体节点列表。
- relationship_ids：属于社区的关系列表。
- text_unit_ids：与社区相关联的文本单元列表。

对应的社区报告如下所示：

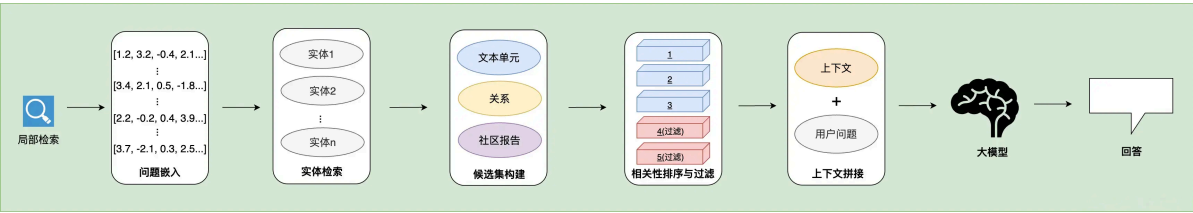
id	human_readable_id	community	level	title	summary
1ded1bd0b	0	0	0	知澜多模态大模型发布	在2025年春季科技大会上，人工智能研究员小明与清北大学计算机学院合作，发布了“知澜”多模态大模型。该模型在自然语言处理、图像识别和语音交互方面表现出色，能够实现跨模态知识融合与智能推理。

- id：社区报告编号，保证全局唯一。
- human_readable_id：便于人类理解或引用的社区报告编号，在展示时更直观，但不一定保证全局唯一。
- community：该报告所属社区的编号。
- level：报告所属社区的层级。
- title：社区报告的简短标题。
- summary：社区报告总结内容。

3.3 检索

GraphRAG 主要提供了两种查询方法，分别为**局部检索**和**全局检索**，局部检索一般会针对问题相关的部分内容进行检索，而全局检索则针对整个知识图谱或所有社区的内容进行检索，类比而言，局部检索就像是在“书的一章里找答案”，而全局检索则更像是在“整本书里找答案”。

3.3.1 局部检索

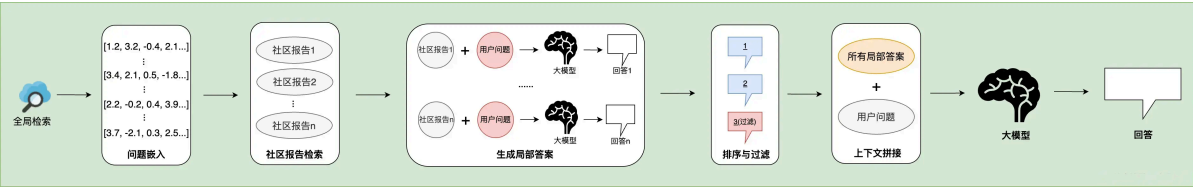


局部检索的流程如下：

1. 问题嵌入：通过 Embedding 模型将问题向量化，将问题文本转换为一个固定长度的浮点型数组。
2. 实体检索：在知识图谱中寻找与查询向量相似的实体。
3. 候选集构建：从找寻到的实体出发，遍历图并收集构建候选集，候选集内容包括原始文本单元（Text Unit）、实体、关系。
4. 相关性排序与过滤：将候选结果按照相似度分数、重要性等维度进行排序，保留 top-k 的候选集。
5. 上下文拼接：将所有筛选得到的候选集拼接成上下文，与用户的问题拼接起来，共同构成提示词（Prompt）。

6. 生成最终答案：将构成的提示词交给大模型处理，并生成最终的结果。

3.3.2 全局检索



全局检索采用了类似于 MapReduce 的策略来完成检索任务，流程如下：

1. 问题嵌入：与局部检索类似，也是通过 Embedding 模型将问题向量化，将问题文本转换成一个固定长度的浮点型数组。
2. 社区报告检索：检索出与查询向量相似的社区报告。
3. 生成局部答案：对每个社区摘要，使用 LLM 生成局部答案，输入的提示词会包括社区摘要内容和用户问题。LLM 除了会生成答案，还会生成对答案的评分，以评估其相关性。这个过程类似于 MapReduce 中的 Map 阶段。
4. 排序与过滤：通过先前的分数来对生成的社区级答案进行排序，并过滤掉分数过低的答案。
5. 上下文拼接：将之前过滤得到的所有局部答案拼接成上下文，与用户的问题拼接起来，共同构成提示词（Prompt）。这个过程类似于 MapReduce 的 Reduce 阶段。
6. 生成最终答案：将构成的提示词交给大模型处理，并生成最终的结果。

3.3.3 局部检索 or 全局检索

	局部检索	全局检索
适用场景	针对特定实体或事件的具体细节查询	需要整体理解整个数据集的抽象性问题
数据处理方式	基于实体进行查询，利用图遍历构建上下文	使用预先计算的社区摘要进行查询
查询方式	从最相关的实体开始，逐步构建上下文以生成回答	对每个社区生成独立的回答，然后将相关回答汇总成最终结果
计算效率	计算量较小，适合快速响应	可能需要处理大量社区摘要，计算资源消耗较大
适用数据规模	适用于小规模数据集，关注具体细节	适用于大规模数据集，能够提供全局视角
优点	计算效率高，响应快速	提供全面的视角
缺点	可能忽略全局上下文	计算资源消耗大，可能导致信息冗余

四、GraphRAG 实战

4.1 前置环境搭建

在使用 GraphRAG 之前，需要先安装对应的环境：

```
# python version = 3.10-3.12
conda create -n graphrag python=3.12
conda activate graphrag

pip install graphrag
```

配置数据集

文件结构（GraphRAG 在索引构建时默认会访问 input 文件夹下的文件，所以数据集一般放在 input 文件夹下，顶层文件夹可以自定义，后续的命令中都修改为对应的目录名即可）：

```
graphrag_demo
├── input
│   └── 1.txt
```

然后初始化 GraphRAG 项目：

```
graphrag init --root ./
```

执行完后，目录中会变成以下内容：

```
graphrag_demo
├── input
│   └── 1.txt
├── prompts
│   ├── basic_search_system_prompt.txt
│   ├── community_report_graph.txt
│   ├── community_report_text.txt
│   ├── drift_reduce_prompt.txt
│   ├── drift_search_system_prompt.txt
│   ├── extract_claims.txt
│   ├── extract_graph.txt
│   ├── global_search_knowledge_system_prompt.txt
│   ├── global_search_map_system_prompt.txt
│   ├── global_search_reduce_system_prompt.txt
│   ├── local_search_system_prompt.txt
│   ├── question_gen_system_prompt.txt
│   └── summarize_descriptions.txt
├── settings.yaml
└── .env
```

- prompts：存放 GraphRAG 内置的提示词模板，对应不同阶段的任务，如 extract_graph.txt 用于抽取实体与关系，构建知识图谱；community_report_text.txt、community_report_graph.txt 用于为每个社区生成报告。
- settings.yaml：配置文件，包含 LLM 配置（模型、温度）、索引参数（chunk 大小、阈值）等。
- .env：环境变量文件，存放密钥、连接配置等信息。

注意

从 2.6.0 版本开始，GraphRAG 支持使用 **LiteLLM** 而不是 `fnllm` 来调用语言模型。LiteLLM 支持 100 多个模型，但需要注意的是，在选择模型时，它必须支持返回符合 **JSON 模式的结构化输出**。

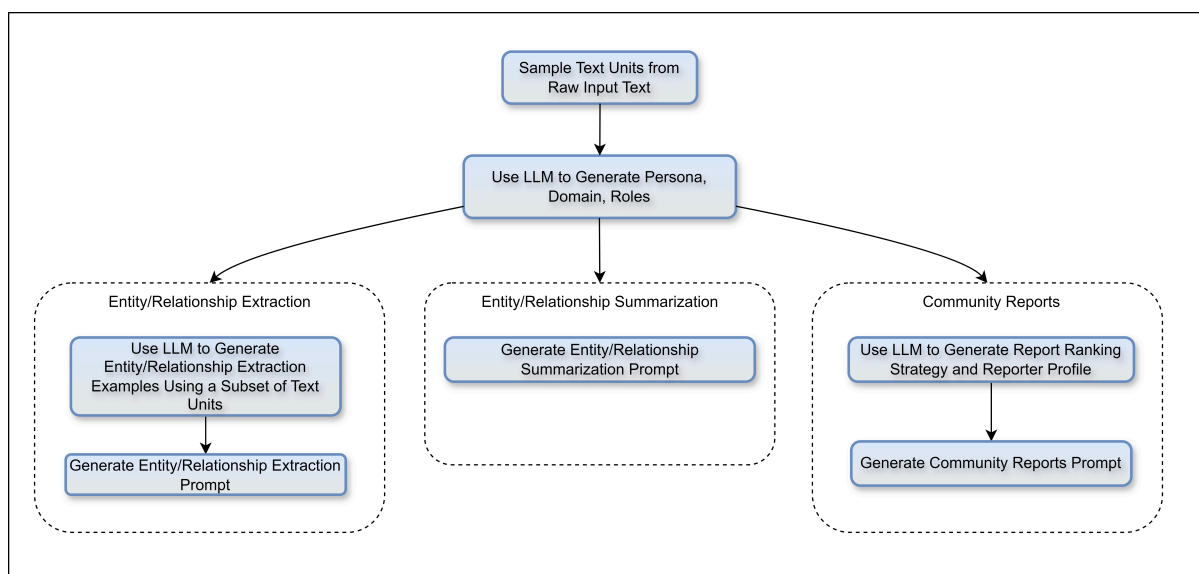
```
models:
  default_chat_model:
    type: chat
    model_provider: openai
    auth_type: api_key
    api_key: ${GRAPHRAG_API_KEY}
    model: gpt-5-mini
    api_base: http://localhost:4000
    model_supports_json: true

  default_embedding_model:
    type: embedding
    model_provider: openai
    auth_type: api_key
    api_key: ${GRAPHRAG_API_KEY}
    model: text-embedding-3-small
    api_base: http://localhost:4000
```

4.2 提示词调优

GraphRAG 提供了创建领域自适应提示词的能力，用于生成知识图谱。这一步是可选的，但强烈建议运行它，因为它将在执行索引运行时产生更好的结果。

这些提示词是通过加载输入、将其分割成块（文本单元），然后运行一系列 LLM 调用和模板替换来生成最终提示词而产生的。



4.2.1 先决条件

在运行自动调优之前，请确保您已使用 `graphrag init` 命令初始化了工作区。这将创建必要的配置文件和默认提示词。

4.2.2 用法

```
python -m graphrag prompt-tune --config ./settings.yaml --root ./ --language
Chinese --output ./prompts
```

4.2.3 命令行选项

- `--config` (必填)：配置文件的路径。这是加载数据和模型设置所必需的。
- `--root` (可选)：数据项目根目录，包括配置文件 (YML、JSON 或 .env) 。默认为当前目录。
- `--domain` (可选)：与您的输入数据相关的领域，例如“空间科学”、“微生物学”或“环境新闻”。如果留空，将从输入数据中推断领域。
- `--selection-method` (可选)：选择文档的方法。选项有 all、random、auto 或 top。默认为 random。
- `--limit` (可选)：使用 random 或 top 选择时要加载的文本单元限制。默认为 15。
- `--language` (可选)：用于输入处理的语言。如果与输入的语言不同，LLM 将进行翻译。默认为 `""`，表示将从输入中自动检测。
- `--max-tokens` (可选)：提示词生成的最大 token 计数。默认为 2000。
- `--chunk-size` (可选)：用于从输入文档生成文本单元的 token 大小。默认为 200。
- `--n-subset-max` (可选)：使用自动选择方法时要嵌入的文本块数量。默认为 300。
- `--k` (可选)：使用自动选择方法时要选择的文档数量。默认为 15。
- `--min-examples-required` (可选)：实体提取提示词所需的最小示例数。默认为 2。
- `--discover-entity-types` (可选)：允许 LLM 自动发现和提取实体。当您的数据涵盖大量主题或高度随机化时，我们建议使用此选项。
- `--output` (可选)：保存生成提示词的文件夹。默认为“prompts”。

4.2.3 调优日志

```
2026-01-12 22:34:50.0686 - INFO - graphrag.api.prompt_tune - Chunking documents...
2026-01-12 22:34:50.0686 - INFO - graphrag.storage.file_pipeline_storage - Creating file storage at D:\Kevin\Workspace\graphrag_demo\input
2026-01-12 22:34:50.0686 - INFO - graphrag.index.input_factory - loading input from root_dir=D:\Kevin\Workspace\graphrag_demo\input
2026-01-12 22:34:50.0686 - INFO - graphrag.index.input_factory - Loading Input InputFileType.text
2026-01-12 22:34:50.0686 - INFO - graphrag.storage.file_pipeline_storage - search D:\Kevin\Workspace\graphrag_demo\input for files matching *.*.txt$
2026-01-12 22:34:50.0697 - INFO - graphrag.index.input.util - Found 6 InputFileType.text files, loading 6
2026-01-12 22:34:50.0697 - INFO - graphrag.index.input.util - Total number of unfiltered InputFileType.text rows: 6
2026-01-12 22:34:50.0697 - INFO - graphrag.index.workflows.create_base_text_units - Starting chunking process for 6 documents
2026-01-12 22:34:50.0799 - INFO - graphrag.index.workflows.create_base_text_units - chunker progress: 1/6
2026-01-12 22:34:50.0799 - INFO - graphrag.index.workflows.create_base_text_units - chunker progress: 2/6
2026-01-12 22:34:50.0799 - INFO - graphrag.index.workflows.create_base_text_units - chunker progress: 3/6
2026-01-12 22:34:50.0807 - INFO - graphrag.index.workflows.create_base_text_units - chunker progress: 4/6
2026-01-12 22:34:50.0808 - INFO - graphrag.index.workflows.create_base_text_units - chunker progress: 5/6
2026-01-12 22:34:50.0810 - INFO - graphrag.index.workflows.create_base_text_units - chunker progress: 6/6
2026-01-12 22:34:50.0814 - INFO - graphrag.api.prompt_tune - Retrieving language model configuration...
2026-01-12 22:34:50.0814 - INFO - graphrag.api.prompt_tune - Creating language model...
2026-01-12 22:34:50.0814 - INFO - graphrag.api.prompt_tune - Generating domain...
2026-01-12 22:34:57.0898 - INFO - graphrag.api.prompt_tune - Generating persona...
2026-01-12 22:34:59.0266 - INFO - graphrag.api.prompt_tune - Generating community report ranking description...
2026-01-12 22:35:04.0372 - INFO - graphrag.api.prompt_tune - Generating entity types...
2026-01-12 22:35:08.0150 - INFO - graphrag.api.prompt_tune - Generating entity relationship examples...
2026-01-12 22:35:59.0080 - INFO - graphrag.api.prompt_tune - Generating entity extraction prompt...
2026-01-12 22:35:59.0086 - INFO - graphrag.api.prompt_tune - Generating entity summarization prompt...
2026-01-12 22:35:59.0086 - INFO - graphrag.api.prompt_tune - Generating community reporter role...
2026-01-12 22:36:04.0247 - INFO - graphrag.api.prompt_tune - Generating community summarization prompt...
2026-01-12 22:36:04.0247 - INFO - graphrag.cli.prompt_tune - Writing prompts to D:\Kevin\Workspace\graphrag_demo\prompts
2026-01-12 22:36:04.0248 - INFO - graphrag.cli.prompt_tune - Prompts written to D:\Kevin\Workspace\graphrag_demo\prompts
```

4.3 索引构建

索引构建工作流: [数据流 - GraphRAG 文档](#)

⚠ 警告

GraphRAG索引操作可能耗费巨大, 请仔细阅读所有文档以了解相关流程及成本, 并从小规模开始尝试。

使用下面的命令构建索引

```
graphrag index --root ./
```

执行完后目录会多出以下内容:

```
graphrag_demo
├── cache
│   ├── community_reporting
│   ├── extract_graph
│   ├── summarize_descriptions
│   └── text_embedding
├── logs
│   └── logs.txt
└── output
    ├── communities.parquet
    ├── community_reports.parquet
    ├── context.json
    ├── documents.parquet
    ├── entities.parquet
    ├── lancedb
    ├── relationships.parquet
    ├── stats.json
    ├── text_units.parquet
    └── summarize_descriptions.txt
```

- cache: 索引过程中调用 LLM 或 Embedding API 的中间结果缓存, 避免重复调用, 如 community_reporting 用于生成社区报告时的中间结果; extract_graph 用于存储从文本抽取出的实体、关系的中间结果。
- logs: 索引过程中的运行日志。
- output: 最终构建的知识索引, 如 communities.parquet 存放社区的结构数据, 记录每个社区包含哪些实体、文本单元等信息; text_units.parquet 存放切分后的文本单元结构数据, 记录 Embedding 信息。

可以使用 pandas 库来解析 parquet 格式的文件内容:

```
# parquet_test.py

import pandas as pd
pd.set_option('display.max_columns', None) # 显示所有列
df = pd.read_parquet('./output/text_units.parquet')
print(df)
```

4.4 检索

4.4.1 局部检索

```
graphrag query --root ./ --method local --query "谁是唐僧，他的主要关系是什么?"
```

唐僧是《西游记》中的主要角色之一，是一位来自东土大唐的高僧。他受观音菩萨的指点，前往西天取经。作为取经队伍的领导者，唐僧一路上遇到了各种困难和妖怪，并且展现了对佛教的虔诚和执着 [Data: Entities (1)]。

唐僧的主要关系包括他与孙悟空的师徒关系。孙悟空是被压在五行山下的神猴，后来成为唐僧的大徒弟。唐僧通过观音菩萨传授的紧箍咒来控制孙悟空的行为，确保他能够听从自己的话。两人之间的关系非常密切，孙悟空不仅保护唐僧免受伤害，还帮助解决了诸如白龙的问题 [Data: Relationships (0, 31, 42); Sources (4, 5)]。

另外，唐僧还与白龙马有着重要的关系。白龙马是唐僧的坐骑，一匹能够变化成马的龙。它原本是西海龙王的儿子，后来被观音菩萨安排成为唐僧的坐骑。白龙马在旅程中帮助唐僧更轻松地行走 [Data: Relationships (5); Entities (2)]。

唐僧还与观音菩萨有密切的关系。观音菩萨在唐僧师徒西天取经的过程中扮演了重要的角色，不仅劝说孙悟空成为唐僧的徒弟，还在唐僧遇到困难时多次伸出援手。观音菩萨传授了紧箍咒给唐僧，帮助他管理和引导孙悟空 [Data: Relationships (52); Sources (4)]。

此外，唐僧还与其他角色有重要的互动，如刘伯钦。刘伯钦是一位猎户，在唐僧处于困境时伸出援手，不仅邀请唐僧到家中作客，还护送唐僧上路，并协助唐僧释放了被封印的孙悟空 [Data: Relationships (30); Sources (2, 3)]。

这些关系共同构成了唐僧在《西游记》中的重要角色和故事背景 [Data: Entities (1); Relationships (0, 31, 42, 52, 5, 30); Sources (2, 3, 4, 5)]。

4.4.2 全局检索

```
graphrag query --root ./ --method global --query "这个故事的主题是什么?"
```

这个故事的主题是关于长安与大唐的西行起点，特别是唐三藏师徒四人从大唐出发西天取经的背景和起点 [Data: Reports (32)]。

唐三藏师徒四人的西行取经之旅是中国古代文学中一个非常著名的故事，主要讲述了唐三藏（玄奘法师）和他的三位徒弟孙悟空、猪八戒和沙僧，为了取得佛教经典，历经九九八十一难，最终成功取得真经的故事。这个故事不仅展现了师徒四人之间的深厚情谊和不屈不挠的精神，还反映了古代丝绸之路上的文化交流和佛教传播 [Data: Reports (32)]。

长安作为大唐的首都，是师徒四人西行取经的起点。长安不仅是当时的政治、经济和文化中心，也是丝绸之路的重要起点之一。从长安出发，师徒四人踏上了漫长而艰难的旅程，途中经历了各种各样的挑战和考验，最终完成了取经的使命 [Data: Reports (32)]。

这个故事的主题不仅体现了师徒四人的勇敢和智慧，还展示了古代中国与外界的文化交流和互动。通过这个故事，我们可以更好地理解古代丝绸之路的历史和文化背景，以及佛教在中国的传播与发展 [Data: Reports (32)]。

4.4.3 DRIFT检索

```
graphrag query --root ./ --method drift --query "谁是唐僧，他的主要关系是什么?"
```

唐僧简介

唐僧，又名唐三藏、玄奘法师，是中国古典名著《西游记》中的重要角色之一。他出生于公元602年，卒于公元664年，是中国历史上著名的佛教僧侣和旅行家。唐僧最为人所知的事迹是其西行取经的经历，他于公元629年开始了前往印度求取佛经的旅程，历经重重困难，最终成功带回了大量佛教经典，并在归国后进行了大量的翻译工作，对后世佛教的发展产生了深远影响。

唐僧的主要关系

1. **徒弟们**:

- **孙悟空**: 唐僧的第一个徒弟，是一只被压在五行山下五百年的神猴。孙悟空武艺高强，神通广大，手持金箍棒，多次帮助唐僧脱离险境 [Data: Sources (1, 3, 4, 5, 18, 19, 20)]。
- **猪八戒**: 唐僧的第二个徒弟，原是天宫的天蓬元帅，因犯天条被贬下凡间。猪八戒性格贪吃懒惰，但力大无穷，善于使用九齿钉耙作战 [Data: Sources (4, 5, 18, 19, 20)]。
- **沙僧**: 唐僧的第三个徒弟，原是流沙河的妖怪，因犯天条被贬下凡间。沙僧沉稳内敛，忠心耿耿，擅长使用降妖宝杖 [Data: Sources (3, 4, 5, 18, 19, 20)]。

2. **观音菩萨**:

- 观音菩萨是唐僧西行取经的重要指导者和支持者。观音菩萨多次在唐僧遇到困难时出现，给予帮助和指引。例如，在观音院斗宝失袈裟的章节中，观音菩萨帮助唐僧从黑风怪手中夺回了袈裟 [Data: Sources (18, 20, 21)]。

3. **白龙马**:

- 白龙马是唐僧的坐骑，原是西海龙王的儿子，因犯了死罪而被观音菩萨安排成为唐僧的坐骑。在鹰愁涧，白龙马吞吃了唐僧的白马，后来被观音菩萨点化成一匹白马 [Data: Sources (0, 5)]。

4. **唐太宗李世民**:

- 唐太宗是唐僧西行取经的发起者。唐太宗命令唐僧前往西天取经，并赐予他护身袈裟等宝物 [Data: Sources (1)]。

总结

唐僧是《西游记》中的核心人物，他不仅是一位虔诚的佛教僧侣，也是取经队伍的领导者。他的主要关系包括与徒弟们之间的师徒关系，与观音菩萨之间的指导关系，与白龙马之间的伙伴/坐骑关系，以及与唐太宗之间的君臣关系。这些关系共同构建了《西游记》中唐僧西行取经的宏大叙事。

4.5 知识图谱索引增量更新

4.5.1 准备增量测试文档

1. 将原 `/input` 文件夹下 `1-6.txt` 文件删除
2. 将 `7-9.txt` 文件直接放入 `/input` 文件夹下

4.5.2 修改配置参数

修改 `settings.yaml` 文件内容，进行增量索引构建，则开启该参数设置

```
update_index_output:
  type: file
  base_dir: "update_output"
```

4.5.3 构建索引

```
graphrag index --root ./
```

4.5.4 知识图谱可视化

打开 `neo4j_test.py` 脚本，修改文件所在路径为存储增量数据的文件路径

```
GRAPHRAG_FOLDER = "./update_output"
```

五、GraphRAG 可视化

通过将 GraphRAG 索引阶段产生的 parquet 文件导入至图数据库 neo4j，可以实现 GraphRAG 可视化的效果，让我们看到 GraphRAG 构建的整个知识图谱样貌。

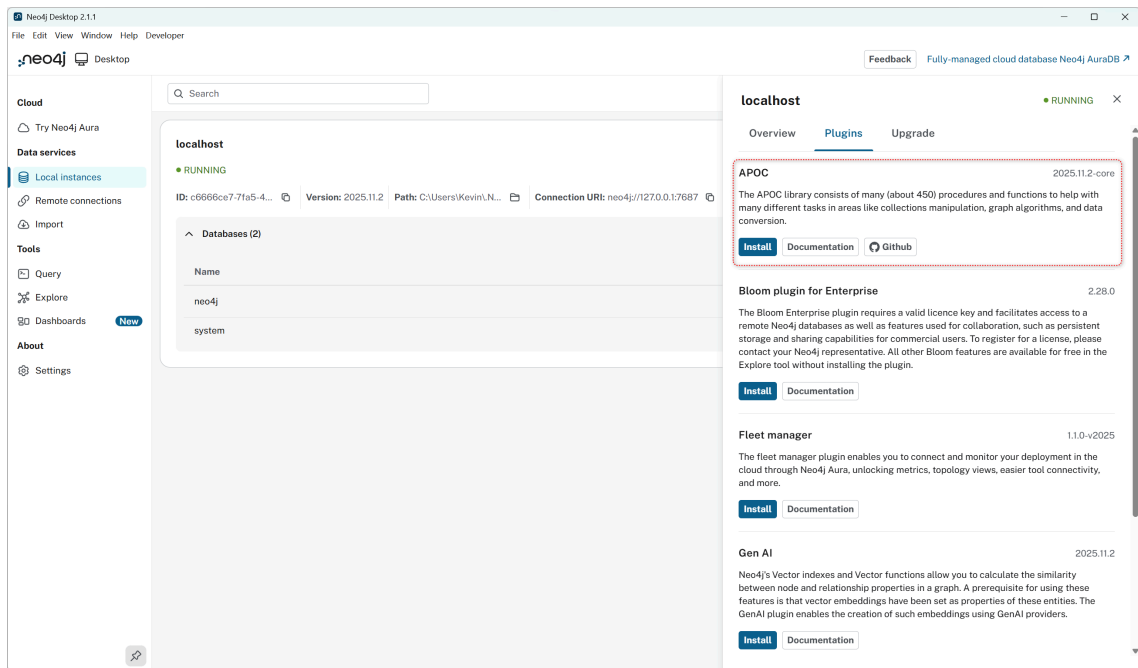
说明：

neo4j 是一个高性能的图数据库，专门用来存储、管理和查询图数据。它以“节点”和“关系”为核心概念，将实体和实体之间的连接关系进行建模，使得复杂的关联数据可以自然、直观地表示和高效操作。

[Neo4j 中文文档](#)

首先需要在本机上安装 neo4j，mac 可以使用 brew 直接安装：brew install neo4j，此外还需要安装 APOC 插件（参考[Installation - APOC Core Documentation](#)）。

1. 下载并安装 Neo4j for Desktop
下载链接：[Neo4j Desktop Download](#)
2. 创建本地数据库
3. 安装APOC插件



4. 然后导入 parquet 文件到 neo4j, 而由于 neo4j 本身不支持直接导入这种格式的文件, 因此我们可以先使用 python 的 pandas 库读取 parquet 文件, 转换成 DataFrame 后再批量导入至 neo4j。具体做法如下:

```
# neo4j_test_1.py
# 实现了完整的图数据构建流程:
# 1. 导入原始文档 (可选)
# 2. 创建文本块节点并连接到文档
# 3. 识别并导入实体, 连接到相应的文本块
# 4. 建立实体间的关系
# 5. 将实体组织到社区中
# 6. 为每个社区添加综合报告和发现

import pandas as pd
from neo4j import GraphDatabase
import time

# 用于高效批量导入数据
# 接收Cypher语句、DataFrame和批次大小作为参数
# 将大DataFrame分割成小批次处理, 避免内存溢出
# 使用UNWIND语句在Neo4j中高效处理批量数据
def batched_import(statement, df, batch_size=1000):
    total = len(df)
    start_s = time.time()
    for start in range(0, total, batch_size):
        batch = df.iloc[start: min(start + batch_size, total)]
        result = driver.execute_query("UNWIND $rows AS value " + statement,
                                      rows=batch.to_dict('records'),
                                      database_=NEO4J_DATABASE)

        print(result.summary.counters)
    print(f'{total} rows in {time.time() - start_s} s.')
    return total

# neo4j 环境配置
NEO4J_URI = "neo4j://127.0.0.1:7687"
NEO4J_USERNAME = "neo4j"
NEO4J_PASSWORD = "88888888"
```

```

NEO4J_DATABASE = "neo4j"

driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USERNAME, NEO4J_PASSWORD))

# parquet 文件存储目录
GRAPHRAG_FOLDER = "./output"

documents = pd.read_parquet(GRAPHRAG_FOLDER + "/documents.parquet")
text_units = pd.read_parquet(GRAPHRAG_FOLDER + "/text_units.parquet")
entities = pd.read_parquet(GRAPHRAG_FOLDER + "/entities.parquet")
relationships = pd.read_parquet(GRAPHRAG_FOLDER + "/relationships.parquet")
communities = pd.read_parquet(GRAPHRAG_FOLDER + "/communities.parquet")
community_reports = pd.read_parquet(GRAPHRAG_FOLDER +
"/community_reports.parquet")

# 导入 documents (可选)
statement = """
MERGE (d:__Document__ {id:value.id})
SET d += value {.title, .text}
"""
batched_import(statement, documents)

# 导入 text_units (文本块)
statement = """
MERGE (c:__Chunk__ {id:value.id})
SET c += value {.text, .n_tokens}
WITH c, value
UNWIND value.document_ids AS document
MATCH (d:__Document__ {id:document})
MERGE (c)-[:PART_OF]->(d)
"""
batched_import(statement, text_units)

# 导入 entities (实体)
entity_statement = """
MERGE (e:__Entity__ {id:value.id})
SET e += value {.human_readable_id, .description,
name:replace(value.title,' ','')}
WITH e, value
CALL apoc.create.addLabels(e, case when coalesce(value.type,'') = '' then [] else
[apoc.text.upperCamelCase(replace(value.type,' ',''))] end) yield node
UNWIND value.text_unit_ids AS text_unit
MATCH (c:__Chunk__ {id:text_unit})
MERGE (c)-[:HAS_ENTITY]->(e)
"""
batched_import(entity_statement, entities)

# 导入 relationships (实体间关系)
relationships_statement = """
MATCH (source:__Entity__ {name:replace(value.source,' ','')})
MATCH (target:__Entity__ {name:replace(value.target,' ','')})
MERGE (source)-[rel:RELATED {id: value.id}]->(target)
SET rel += value {.weight, .human_readable_id, .description, .text_unit_ids}
RETURN count(*) as createdReIs
"""
batched_import(relationships_statement, relationships)

```

```

# 导入 communities（社区）
# 1. 创建或合并__Community__节点（代表实体的聚类社区）
# 2. 通过遍历关系ID，将相关实体连接到社区
# 3. 为社区内的每个实体创建IN_COMMUNITY关系
communities_statement = """
MERGE (c:__Community__ {id:value.id})
SET c += value {.level, .title, .community}
/*
UNWIND value.text_unit_ids as text_unit_id
MATCH (t:__Chunk__ {id:text_unit_id})
MERGE (c)-[:HAS_CHUNK]->(t)
WITH distinct c, value
*/
WITH *
UNWIND value.relationship_ids as rel_id
MATCH (start:__Entity__)-[:RELATED {id:rel_id}]->(end:__Entity__)
MERGE (start)-[:IN_COMMUNITY]->(c)
MERGE (end)-[:IN_COMMUNITY]->(c)
RETURN count(distinct c) as createdCommunities
"""
batched_import(communities_statement, communities)

# 导入 community_reports（社区报告）
# 1. 匹配或创建社区节点
# 2. 设置社区详细属性，包括排名、解释、完整内容和摘要
# 3. 为报告中的每个发现(Finding)创建节点
# 4. 建立社区与发现之间的HAS_FINDING关系
community_reports_statement = """
MERGE (c:__Community__ {community:value.community})
SET c += value {.level, .title, .rank, .rank_explanation, .full_content,
.summary}
WITH c, value
UNWIND range(0, size(value.findings)-1) AS finding_idx
WITH c, value, finding_idx, value.findings[finding_idx] as finding
MERGE (c)-[:HAS_FINDING]->(f:Finding {id:finding_idx})
SET f += finding
"""
batched_import(community_reports_statement, community_reports)

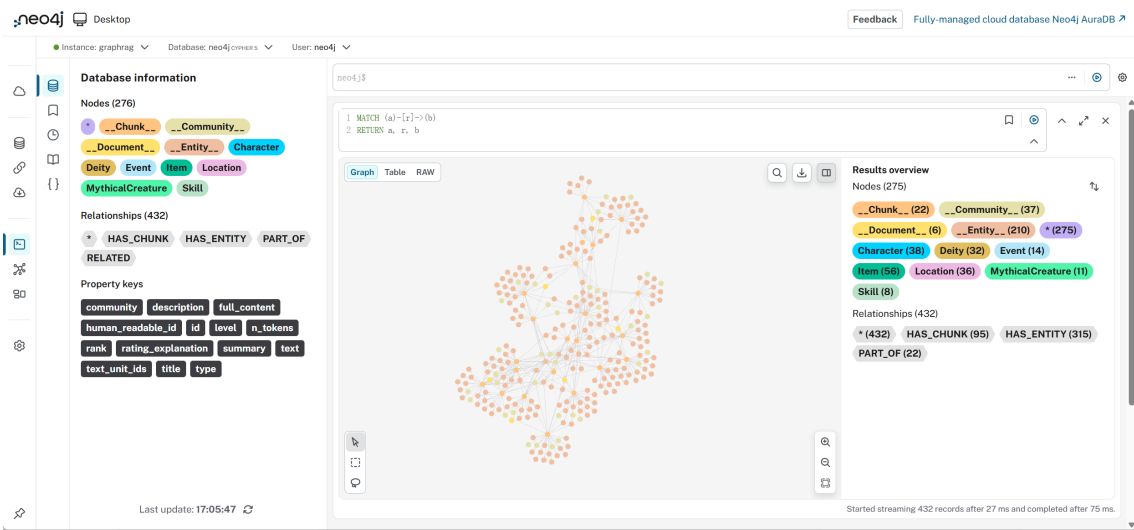
```

5. 查询所有的节点和关系

```

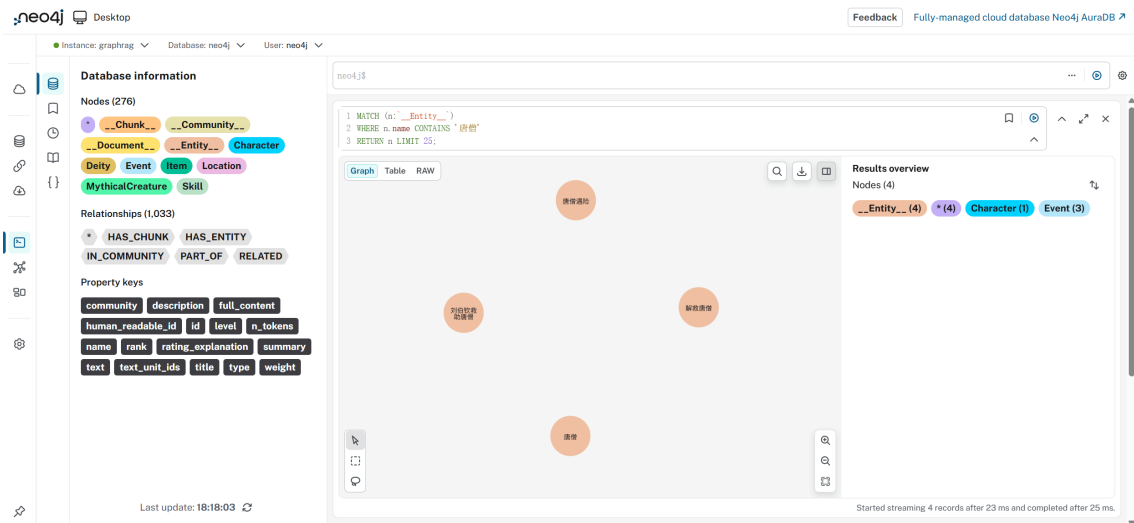
MATCH (a)-[r]->(b)
RETURN a, r, b

```



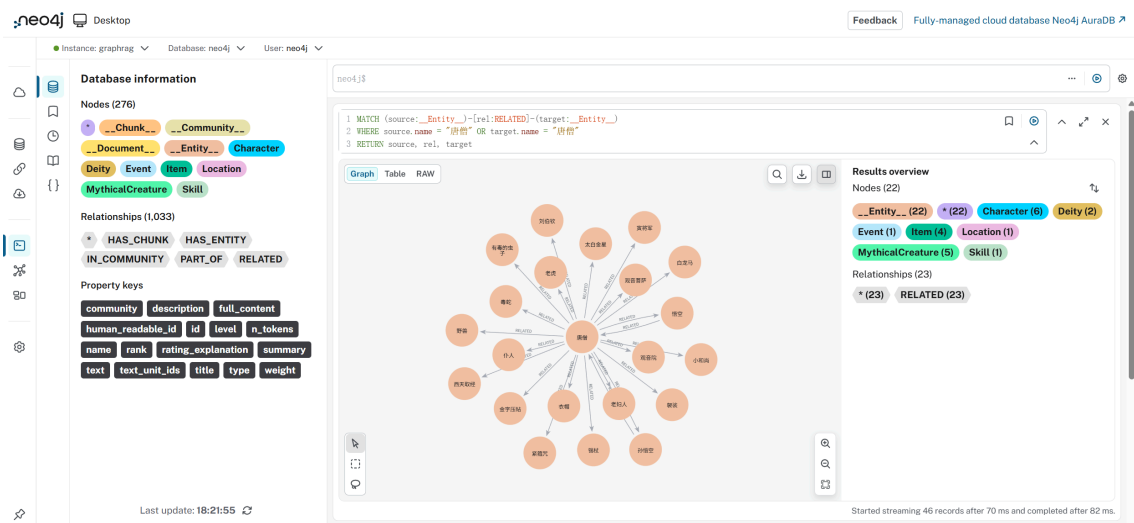
6. 查询包含“唐僧”的所有实体

```
MATCH (n: `__Entity__`)
WHERE n.name CONTAINS '唐僧'
RETURN n LIMIT 25;
```



7. 查询“唐僧”实体关联的所有实体

```
MATCH (source: __Entity__) -[rel: RELATED] - (target: __Entity__)
WHERE source.name = "唐僧" OR target.name = "唐僧"
RETURN source, rel, target
```



六、Neo4j GraphRAG

[neo4j-graphrag-python 文档](#)

七、LlamaIndex 实现 GraphRAG

[GraphRAG 在 LlamaIndex 中的实现 - V2 - LlamaIndex 框架](#)

[LlamaIndex - 开发者指南 - Neo4j 文档](#)

八、LangChain 实现 GraphRAG

[Graph RAG - Docs by LangChain](#)

[LangChain Neo4j 集成 - 开发者指南 - Neo4j 文档](#)

```
> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (c:Character {name: '唐僧'})
OPTIONAL MATCH (c)-[r]->(o)
WHERE type(r) IN ['RELATED','IN_COMMUNITY']
RETURN c { .id, .name, .description, .text_unit_ids } AS character,
        collect(DISTINCT { relation: type(r), rel_props: properties(r),
        node_labels: labels(o), node_id: o.id, node_name: o.name }) AS relationships;
Full Context:
```

```
[{'character': {'description': '唐僧是《西游记》中的主要角色之一，是一位来自东土大唐的高僧，受观音菩萨指点前往西天取经。作为取经队伍的领导者，他一路上遇到了各种困难和妖怪。唐僧不仅是孙悟空的师父，还在孙悟空离开后学会了紧箍咒，以此来控制孙悟空的行为。在故事中，他曾通过念紧箍咒来教训孙悟空。唐僧是一位虔诚向佛的和尚，对取经事业非常执着。有一次，他的袈裟被黑风怪偷走，这让他感到非常焦虑。此外，唐僧还是白龙马的新主人，在这次事件中险些被白龙攻击。尽管面临种种挑战，唐僧依然坚持西行，最终成功完成了取经的使命。', 'id': 'b50db866-ebef-4fa4-a668-360d2ce06b1f', 'name': '唐僧', 'text_unit_ids': ['4f7dd6e2211ec45c76429b9b7a00e3399a4cf56a206c60dd3dec5210165f1db1a654d272abf9b0e39cd534df6816376267c363251ed7c4b6cd830dcb3414848', '323dd29ae246cb133042a3fe876af726901fc67e89e777a77256a72605ca6a6c73f1475568e78bb21e2d0833459508517ee2621bdd57112232da61a4e9f447d5', 'd4a7e570d726608523997fed71c7091e6b6fa6f0c257db4bef9f099a38be47080acb9a3ae58207e8936f8867736bafdd3af1cfd24e67b08acfcbe8edebaff', '15be30841288751cf113b9038888de08c90ce8ee0563e75b254871f76e61845b3948c5e75ff635f11e2d8b9e47fd6a249cf06917e47bba8159375c8f6cc03b2', '3ac4968b98d8de6c83f57decccbaed0e86397757d00506c9feb2394c17bd7b4284a34ddda1c0f9b79381f25137c8236f91a7676ebad114025f4ff15ecb07b797', '73d316b96ce83cf496aca54129df4886da76917175445bb6569f45c55798c5ee6841dce1210275a82eead1068c5638282606f61df545399b38138ab217ec6914', 'f801cf3ab26531b55c18dce094a93c03969bbbc05e69a66b1a2ed537098debd8e710f9b7ba9e3d0d6026bacc97b382bcd5f69ef25710c1162d204af743add9', 'bc5efa981472a7a341a5dc64a4e44d05cc2272887870eb2e809111ef2e42a2473ae1de1f53b0e1c86524397147526aca5dc1562d66494c7aa75baa4f018daa6b']}, 'relationships': [{'node_id': 'ef8d4d9c-299d-4da8-a25f-bab623d209df', 'node_labels': ['__Entity__', 'Character'], 'node_name': '悟空', 'rel_props': {'human_readable_id': 199, 'text_unit_ids': ['73d316b96ce83cf496aca54129df4886da76917175445bb6569f45c55798c5ee6841dce1210275a82eead1068c5638282606f61df545399b38138ab217ec6914'], 'description': '悟空是唐僧的徒弟，保护唐僧免受伤害。', 'weight': 10.0, 'id': '36f15242-bd9d-4e28-addd-e4d3b0736b47'}, 'relation': 'RELATED'}, {'node_id': '603c5827-a367-4a9a-bf69-a0f30d7c1072', 'node_labels': ['__Entity__', 'MythicalCreature'], 'node_name': '白龙马', 'rel_props': {'human_readable_id': 5, 'text_unit_ids': ['4f7dd6e2211ec45c76429b9b7a00e3399a4cf56a206c60dd3dec5210165f1db1a654d272abf9b0e39cd534df6816376267c363251ed7c4b6cd830dcb3414848', '73d316b96ce83cf496aca54129df4886da76917175445bb6569f45c55798c5ee6841dce1210275a82eead1068c5638282606f61df545399b38138ab217ec6914'], 'description': '白龙马成为了唐僧的新坐骑，帮助唐僧继续取经之旅，并在旅途中使唐僧的行走变得更加轻松。', 'weight': 18.0, 'id': 'a0bf8aac-dfd4-4136-a5d4-c53a3fbb111c'}, 'relation': 'RELATED'}, {'node_id': 'ff91d547-ba32-4f41-8823-40281142e941', 'node_labels': ['__Entity__', 'Deity'], 'node_name': '观音菩萨', 'rel_props': {'human_readable_id': 52, 'text_unit_ids': ['d4a7e570d726608523997fed71c7091e6b6fa6f0c257db4bef9f099a38be47080acb9a3ae58207e8936f8867736bafdd3af1cfd24e67b08acfcbe8edebaff', '15be30841288751cf113b9038888de08c90ce8ee0563e75b254871f76e61845b3948c5e75ff635f11e2d8b9e47fd6a249cf06917e47bba8159375c8f6cc03b2', '3ac4968b98d8de6c83f57decccbaed0e86397757d00506c9feb2394c17bd7b4284a34ddda1c0f9b79381f25137c8236f91a7676ebad114025f4ff15ecb07b797'], 'description': '观音菩萨在唐僧师徒西天取经的过程中扮演了重要的角色。她不仅劝说孙悟空成为唐僧的徒弟，一同前往西天取得真经，还在唐僧遇到困难时多次伸出援手。为了让唐僧能够更好地管理和引导孙悟空，观音菩萨传授了紧箍咒给唐僧，这一法术成为了唐僧控制孙悟空行为的有效工具。在孙悟空离开唐僧后，观音菩萨再次帮助唐僧找回了孙悟空，并提供了紧箍咒作为控制孙悟空的手段，确保取经之旅能够顺利进行。', 'weight': 29.0, 'id': '7ba64a9b-7955-49f0-9d53-7b190543e553'}, 'relation': 'RELATED'}, {'node_id': '223a8aab-5dd1-4e1c-bf5a-e85b3233c65e', 'node_labels': ['__Entity__', 'Character'], 'node_name': '寅将军', 'rel_props': {'human_readable_id': 28, 'text_unit_ids': ['323dd29ae246cb133042a3fe876af726901fc67e89e777a77256a72605ca6a6c73f1475568e78bb21e2d0833459508517ee2621bdd57112232da61a4e9f447d5'], 'description': '寅将军捉住了唐
```

僧和他的仆人，还吃掉了唐僧的两个仆人。', 'weight': 8.0, 'id': '59f253d4-f4a9-41bf-b5e3-29ba12d591ff'}, 'relation': 'RELATED'}, {'node_id': '5b92ae80-8083-40e5-bdb7-dca0d4f3d752', 'node_labels': ['__Entity__', 'Deity'], 'node_name': '太白金星', 'rel_props': {'human_readable_id': 29, 'text_unit_ids': ['323dd29ae246cb133042a3fe876af726901fc67e89e777a77256a72605ca6a6c73f1475568e78bb21e2d0833459508517ee2621bdd57112232da61a4e9f447d5'], 'description': '太白金星化身为老人解救了唐僧，并指引他离开危险的地方。', 'weight': 9.0, 'id': '0856611e-45f6-4529-aa7f-863c72c59937'}, 'relation': 'RELATED'}, {'node_id': 'dcf24b37-29cd-443c-b3ec-fe59460625cc', 'node_labels': ['__Entity__', 'Character'], 'node_name': '刘伯钦', 'rel_props': {'human_readable_id': 30, 'text_unit_ids': ['323dd29ae246cb133042a3fe876af726901fc67e89e777a77256a72605ca6a6c73f1475568e78bb21e2d0833459508517ee2621bdd57112232da61a4e9f447d5', 'd4a7e570d726608523997fed71c7091e6b6fa6f0c257db4bef9f099a38be47080acbfa9a3ae58207e8936f8867736bafdd3af1cfd24e67b08acfcbe8edebaff'], 'description': '刘伯钦在唐僧的西行取经之路上扮演了重要的角色。他不仅帮助唐僧从危险中解救出来，还邀请唐僧到自己家中作客。此外，刘伯钦还协助唐僧释放了被封印的孙悟空，并见证了这一重要的过程。这些行为体现了刘伯钦的善良和勇敢，也为唐僧的旅程提供了关键的支持。', 'weight': 13.0, 'id': '5d5b16ae-ac9c-4c39-a8dd-dc912f61eb89'}, 'relation': 'RELATED'}, {'node_id': 'c83600d6-ceac-4221-9f1c-7e70a38f2608', 'node_labels': ['__Entity__', 'Character'], 'node_name': '孙悟空', 'rel_props': {'human_readable_id': 31, 'text_unit_ids': ['323dd29ae246cb133042a3fe876af726901fc67e89e777a77256a72605ca6a6c73f1475568e78bb21e2d0833459508517ee2621bdd57112232da61a4e9f447d5'], 'description': '孙悟空是被压在两界山下的神猴，后来成为唐僧的弟子。', 'weight': 10.0, 'id': '4fc9bc53-1843-41e2-b8ed-98d0ae6217c7'}, 'relation': 'RELATED'}, {'node_id': '99c7ecb2-940a-4372-8214-264089dba808', 'node_labels': ['__Entity__', 'MythicalCreature'], 'node_name': '老虎', 'rel_props': {'human_readable_id': 33, 'text_unit_ids': ['323dd29ae246cb133042a3fe876af726901fc67e89e777a77256a72605ca6a6c73f1475568e78bb21e2d0833459508517ee2621bdd57112232da61a4e9f447d5'], 'description': '唐僧在双叉岭遇到凶恶的老虎，陷入了危险。', 'weight': 6.0, 'id': '426a76ae-ff3b-4d7c-b0ef-2bd3c108a066'}, 'relation': 'RELATED'}, {'node_id': 'c97aa63e-1734-4682-b7ec-57d3b9224ece', 'node_labels': ['__Entity__', 'MythicalCreature'], 'node_name': '毒蛇', 'rel_props': {'human_readable_id': 34, 'text_unit_ids': ['323dd29ae246cb133042a3fe876af726901fc67e89e777a77256a72605ca6a6c73f1475568e78bb21e2d0833459508517ee2621bdd57112232da61a4e9f447d5'], 'description': '唐僧在双叉岭遇到吐着红信的毒蛇，陷入了危险。', 'weight': 6.0, 'id': '679e6721-8e14-4502-9374-c16e79c3157a'}, 'relation': 'RELATED'}, {'node_id': '78c58853-2725-4425-88cd-2d38d7ebedd4', 'node_labels': ['__Entity__', 'MythicalCreature'], 'node_name': '有毒的虫子', 'rel_props': {'human_readable_id': 35, 'text_unit_ids': ['323dd29ae246cb133042a3fe876af726901fc67e89e777a77256a72605ca6a6c73f1475568e78bb21e2d0833459508517ee2621bdd57112232da61a4e9f447d5'], 'description': '唐僧在双叉岭遇到有毒的虫子，陷入了危险。', 'weight': 6.0, 'id': 'f5539c8c-33a9-4938-88cb-27cba861d84f'}, 'relation': 'RELATED'}, {'node_id': '461fc8f1-d22a-4735-8167-2a185b34dc60', 'node_labels': ['__Entity__', 'MythicalCreature'], 'node_name': '野兽', 'rel_props': {'human_readable_id': 36, 'text_unit_ids': ['323dd29ae246cb133042a3fe876af726901fc67e89e777a77256a72605ca6a6c73f1475568e78bb21e2d0833459508517ee2621bdd57112232da61a4e9f447d5'], 'description': '唐僧在双叉岭遇到各种未见过的野兽，陷入了危险。', 'weight': 6.0, 'id': '83393841-c85f-4623-b2db-3a743ef1074f'}, 'relation': 'RELATED'}, {'node_id': 'ff47fbe4-a11d-4b62-aa46-9c8ef08b3711', 'node_labels': ['__Entity__'], 'node_name': '仆人', 'rel_props': {'human_readable_id': 32, 'text_unit_ids': ['323dd29ae246cb133042a3fe876af726901fc67e89e777a77256a72605ca6a6c73f1475568e78bb21e2d0833459508517ee2621bdd57112232da61a4e9f447d5'], 'description': '唐僧的仆人被寅将军杀害，唐僧因此陷入极度危险。', 'weight': 8.0, 'id': '3b1dd085-0427-4518-93e1-d4b401d5c3cf'}, 'relation': 'RELATED'}, {'node_id': '461b40de-f87c-4b60-afda-259b7e0243cf', 'node_labels': ['__Entity__', 'Event'], 'node_name': '西天取经', 'rel_props': {'human_readable_id': 53, 'text_unit_ids':

['d4a7e570d726608523997fed71c7091e6b6fa6f0c257db4bef9f099a38be47080acbfac9a3ae58207e8936f8867736bafdd3af1cfd24e67b08acfcbe8edebaff', '15be30841288751cf113b9038888de08c90ce8ee0563e75b254871f76e61845b3948c5e75ff635f111e2d8b9e47fd6a249cf06917e47bba8159375c8f6cc03b2'], 'description': '唐僧是西天取经的主要人物之一，也是这一史诗般旅程的主角。他带领徒弟们前往西天取经，即使在孙悟空暂时离开的情况下，依然坚定不移地继续前行。在这一过程中，唐僧展现了非凡的毅力和坚定的信仰，成为了取经之路上不可或缺的灵魂人物。', 'weight': 20.0, 'id': 'd0ba7b6d-d696-4eea-b1d4-3e424bf9dd8f'}, 'relation': 'RELATED'}, {'node_id': 'a22a4607-9293-4b63-9b54-dc2fa72c70b5', 'node_labels': ['__Entity__', 'Item'], 'node_name': '金字压帖', 'rel_props': {'human_readable_id': 54, 'text_unit_ids': ['d4a7e570d726608523997fed71c7091e6b6fa6f0c257db4bef9f099a38be47080acbfac9a3ae58207e8936f8867736bafdd3af1cfd24e67b08acfcbe8edebaff'], 'description': '唐僧移除了金字压帖，从而释放了孙悟空。', 'weight': 8.0, 'id': 'dba60fec-f605-4b56-96f6-4f04298443c4'}, 'relation': 'RELATED'}, {'node_id': '2dff059-48ea-4eef-b987-f6d009681579', 'node_labels': ['__Entity__', 'Skill'], 'node_name': '紧箍咒', 'rel_props': {'human_readable_id': 57, 'text_unit_ids': ['15be30841288751cf113b9038888de08c90ce8ee0563e75b254871f76e61845b3948c5e75ff635f111e2d8b9e47fd6a249cf06917e47bba8159375c8f6cc03b2', '3ac4968b98d8de6c83f57decccbaed0e86397757d00506c9feb2394c17bd7b4284a34ddda1c0f9b79381f25137c8236f91a7676ebad114025f4ff15ecb07b797'], 'description': '唐僧学会了紧箍咒，并用它来控制孙悟空的行为，使其感到头痛。通过这种方式，唐僧能够有效地管理孙悟空，确保他在取经的路上遵从指示。紧箍咒是一种特殊的咒语，只有唐僧能够使用，它对孙悟空有着强大的约束力。', 'weight': 26.0, 'id': '4b3261d4-1a55-472f-97c1-9f1e7678ce78'}, 'relation': 'RELATED'}, {'node_id': 'e5dc8671-2d20-4c49-9937-e82ef251cb35', 'node_labels': ['__Entity__', 'Item'], 'node_name': '衣帽', 'rel_props': {'human_readable_id': 58, 'text_unit_ids': ['15be30841288751cf113b9038888de08c90ce8ee0563e75b254871f76e61845b3948c5e75ff635f111e2d8b9e47fd6a249cf06917e47bba8159375c8f6cc03b2'], 'description': '唐僧将观音菩萨送给他的衣帽给了孙悟空，当孙悟空戴上后，唐僧可以念紧箍咒控制他。', 'weight': 1.0, 'id': 'e628e548-a30d-4410-8c9a-283d967866dc'}, 'relation': 'RELATED'}, {'node_id': 'fd1b2b9a-0411-4670-86bb-c23fad12ac0d', 'node_labels': ['__Entity__', 'Item'], 'node_name': '锡杖', 'rel_props': {'human_readable_id': 59, 'text_unit_ids': ['15be30841288751cf113b9038888de08c90ce8ee0563e75b254871f76e61845b3948c5e75ff635f111e2d8b9e47fd6a249cf06917e47bba8159375c8f6cc03b2'], 'description': '唐僧使用锡杖作为行走的辅助工具。', 'weight': 6.0, 'id': 'f52c603d-9852-4f9c-9bd5-0958632b517e'}, 'relation': 'RELATED'}, {'node_id': 'bd3ddee8-47ec-49c9-9e01-ab74e87d9b18', 'node_labels': ['__Entity__', 'Character'], 'node_name': '老妇人', 'rel_props': {'human_readable_id': 60, 'text_unit_ids': ['15be30841288751cf113b9038888de08c90ce8ee0563e75b254871f76e61845b3948c5e75ff635f111e2d8b9e47fd6a249cf06917e47bba8159375c8f6cc03b2'], 'description': '老妇人实际上是观音菩萨的化身，她给唐僧带来了控制孙悟空的紧箍咒和衣帽。', 'weight': 9.0, 'id': 'f5ceff1b-612b-430a-88b9-1c0f94429ae9'}, 'relation': 'RELATED'}, {'node_id': '4376f6c9-9592-420e-8411-ab6795dfd55f', 'node_labels': ['__Entity__', 'Location'], 'node_name': '观音院', 'rel_props': {'human_readable_id': 197, 'text_unit_ids': ['73d316b96ce83cf496aca54129df4886da76917175445bb6569f45c55798c5ee6841dce1210275a82eead1068c5638282606f61df545399b38138ab217ec6914'], 'description': '唐僧师徒在旅途中来到观音院投宿，遇到了老和尚。', 'weight': 7.0, 'id': '6247fb40-8312-4644-9c75-818372a3522b'}, 'relation': 'RELATED'}, {'node_id': '9d8e93dc-fe07-4f0a-abb8-9eac58735869', 'node_labels': ['__Entity__', 'Item'], 'node_name': '袈裟', 'rel_props': {'human_readable_id': 198, 'text_unit_ids': ['73d316b96ce83cf496aca54129df4886da76917175445bb6569f45c55798c5ee6841dce1210275a82eead1068c5638282606f61df545399b38138ab217ec6914'], 'description': '袈裟是唐僧的宝物，被老和尚贪图并试图借走。', 'weight': 9.0, 'id': '78ce2955-7737-4745-8f84-e1f2926b76a0'}, 'relation': 'RELATED'}, {'node_id': 'ae8b0d54-5a9e-432b-b5bb-7fecc91f2b09', 'node_labels': ['__Entity__'], 'node_name': '小和尚', 'rel_props': {'human_readable_id': 200, 'text_unit_ids':

```
[ '73d316b96ce83cf496aca54129df4886da76917175445bb6569f45c55798c5ee6841dce1210275a82eead1068c5638282606f61df545399b38138ab217ec6914'], 'description': '小和尚是观音院中的和尚，试图伤害唐僧。', 'weight': 6.0, 'id': '5bfaa76b-1103-4785-82a8-d626d77a41d5'}, 'relation': 'RELATED'}, {'node_id': None, 'node_labels': ['__Community__'], 'node_name': None, 'rel_props': {}, 'relation': 'IN_COMMUNITY'}]]]
```

> Finished chain.

--- 最终答案 ---

唐僧是《西游记》中的主要角色之一，是一位来自东土大唐的高僧，受观音菩萨指点前往西天取经并带领取经队伍。他是孙悟空的师父（孙悟空为其徒弟并保护唐僧）、白龙马的新主人（作为坐骑帮助行走）、并从观音菩萨处得到并学会了紧箍咒以控制孙悟空。其他重要关系还包括：袈裟（被偷走的宝物）、金字压帖（移除后释放孙悟空）、太白金星与刘伯钦等人在旅途中曾救助或帮助过他。

--- 生成的 Cypher 查询语句 ---

```
MATCH (c:Character {name: '唐僧'})
OPTIONAL MATCH (c)-[r]->(o)
WHERE type(r) IN ['RELATED','IN_COMMUNITY']
RETURN c { .id, .name, .description, .text_unit_ids } AS character,
        collect(DISTINCT { relation: type(r), rel_props: properties(r),
                           node_labels: labels(o), node_id: o.id, node_name: o.name }) AS relationships;
```

九、总结

从 GraphRAG 索引和检索的流程中不难看出，由于其构建了一个很完整的知识图谱，因此在检索时能够更精准地定位与问题相关联的上下文，从而提高生成结果的相关性和准确性。同时，知识图谱结构使得复杂关系和多跳推理成为可能，这在传统基于向量检索的 RAG 方法中往往难以实现。尤其在大规模知识库或跨领域信息整合场景下，GraphRAG 能有效降低信息遗漏的风险，并提升对复杂问题的回答能力，为实际应用提供了更可靠的支持。

然而，正因为知识图谱需要完整的实体与关系构建，这不可避免地带来了额外的成本与复杂性。大规模图谱的维护与更新会增加计算资源的消耗，同时随着图规模的增长，查询效率也可能受到影响。在**微软 GraphRAG 中，知识图谱的生成完全依赖大模型完成**，这意味着整个索引过程会消耗大量的 token，而**图谱生成的质量高度依赖基座大模型本身的能力**，若大模型本身的能力较差，最终生成的知识图谱质量也会较低。并且，若后续有新的内容加入，需要重新绘制整个知识图谱，这会进一步增加成本。

总的来说，尽管 GraphRAG 在构建与维护知识图谱上存在一定成本与复杂性，但随着后续大模型能力的不断提升、图谱自动化生成技术的优化，这些挑战最终有望得到有效缓解。未来，GraphRAG 有潜力在更大规模、更多领域的知识整合中展现出更强的适应性和可靠性，为智能问答、知识推理以及跨模态信息处理等应用提供坚实的技术基础。

附录： LiteLLM



官网：<https://www.litellm.ai/>

LiteLLM 是一个开源的 Python 库和 AI 网关（Proxy Server），其核心功能是将 100 多种不同大型语言模型（LLM）的 API 统一封装成 **OpenAI API 格式**。

核心定位

LiteLLM 被称为 LLM 的“通用遥控器”。无论底层调用的是 Anthropic、Azure、Google Vertex AI 还是本地的 Ollama、vLLM，开发者都可以使用相同的代码逻辑进行调用。

主要产品形态

- **Python SDK**: 通过 `pip install litellm` 安装，直接在代码中使用 `completion()` 函数调用各类模型，支持自动翻译输入/输出格式。
- **代理服务器 (Proxy Server)**: 一个基于 FastAPI 的中转服务，提供统一的终端节点。它支持虚拟 API Key、多租户管理和负载均衡，适合企业级部署。

关键功能与优势

- **统一格式**: 所有模型均支持 OpenAI 的聊天、嵌入（Embedding）和图片生成接口格式。
- **可靠性机制**: 内置自动重试、故障转移（Failover）和负载均衡功能。当某个模型供应商宕机或触发速率限制时，可自动切换到备选模型。
- **成本与用量监控**: 提供实时费用追踪、预算设置和导出功能，可按团队或项目统计支出。
- **可观测性**: 集成了 Langfuse、MLflow、Helicone 等监控工具，支持自定义日志回调。
- **企业级安全**（2026年最新特性）: 支持 SSO 登录、IP 访问控制、机密管理（如 AWS Secrets Manager）以及符合 GDPR 的日志脱敏。

截至 2026 年初，LiteLLM 已实现显著的性能优化，其代理服务器的开销降低了 50%（通过 O(1) 字典查找取代顺序链），处理速度提升了约 92.7%。此外，它还新增了对 Manus API、AWS Polly TTS 等多模态接口的支持。

Docker-Compose 本地部署

```
services:
  litellm:
    build:
      context: .
      args:
        target: runtime
```

```

image: docker.litellm.ai/berriai/litellm:main-stable
#####
## Uncomment these lines to start proxy with a config.yaml file ##
# volumes:
#   - ./config.yaml:/app/config.yaml
# command:
#   - "--config=/app/config.yaml"
#####
ports:
  - "4000:4000" # Map the container port to the host, change the host port if
necessary
environment:
  DATABASE_URL: "postgresql://llmproxy:dbpassword9090@db:5432/litellm"
  STORE_MODEL_IN_DB: "True" # allows adding models to proxy via UI
env_file:
  - .env # Load local .env file
depends_on:
  - db # Indicates that this service depends on the 'db' service, ensuring
'db' starts first
healthcheck: # Defines the health check configuration for the container
  test:
    - CMD-SHELL
    - python3 -c "import urllib.request;
urllib.request.urlopen('http://localhost:4000/health/liveliness')" # Command to
execute for health check
  interval: 30s # Perform health check every 30 seconds
  timeout: 10s # Health check command times out after 10 seconds
  retries: 3 # Retry up to 3 times if health check fails
  start_period: 40s # wait 40 seconds after container start before beginning
health checks

db:
  image: postgres:16
  restart: always
  container_name: litellm_db
  environment:
    POSTGRES_DB: litellm
    POSTGRES_USER: llmproxy
    POSTGRES_PASSWORD: dbpassword9090
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data # Persists Postgres data across
container restarts
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -d litellm -U llmproxy"]
    interval: 1s
    timeout: 5s
    retries: 10

volumes:
  postgres_data:
    name: litellm_postgres_data # Named volume for Postgres data persistence

```

执行安装

```
docker compose up -d
```

浏览器访问

<http://localhost:4000>